**COMMON COURSE ASSIGNMENT**

**CSA08 – GENERATIVE AI & PROMPT ENGINEERING | Covering Large Language Models & Prompt-Driven Engineering Assistance**

## A. Assignment Information

| | |
|---|---|
| **Department** | Computer Science and Engineering |
| **Programme** | B.E. / B.Tech – CSE |
| **Course Code & Course Name** | CSA08 – Generative AI and Prompt Engineering |
| **Academic Year / Batch** | 2026 – 2027 |
| **Faculty Name** | Dr.Chandravathi C |
| **Assignment Title** | Design and Implementation of PromptCraft – A Prompt-Driven Engineering Assistant using Large Language Models and Prompt Engineering |
| **Date of Issue** | 21.08.2026 |
| **Date of Submission** | 03.09.2026 |
| **Maximum Marks** | 100 |
| **Course Outcome(s) – CO** | CO1 & CO2 |
| **Bloom's Taxonomy Level** | L3 – Apply, L4 – Analyze, L5 – Evaluate |
| **SDG Mapping (SDG 1–17)** | SDG 4 – Quality Education; SDG 9 – Industry, Innovation and Infrastructure |
| **Industry / Societal Relevance** | Prompt-driven AI assistants are increasingly used across the software industry to accelerate coding, debugging, documentation, and requirement analysis, making skills in large language models and prompt engineering directly relevant to modern engineering practice. |

## B. Assignment Problem / Challenge

The central challenge is to build an engineering assistant that does more than return a generic answer. It should help a student understand how a prompt is formed, why a particular strategy is selected, and how the generated result can be checked before use. PromptCraft therefore treats the prompt as an explicit engineering artifact rather than an invisible message sent to a model.

The application must support three complementary activities: constructing prompts, solving common software-engineering tasks, and comparing prompting strategies. The design emphasizes traceability, because users should be able to inspect the instructions, context, examples, model response, and simple quality indicators.

## C. Problem Statement

Students and junior developers frequently spend time rewriting prompts when an LLM response is incomplete, poorly formatted, or technically weak. PromptCraft addresses this issue through a lightweight Streamlit application that organizes prompt construction into repeatable templates and provides immediate feedback for engineering-oriented tasks.

### Major Functional Areas

| Area | Purpose | Typical Input |
|---|---|---|
| Prompt Studio | Construct and inspect prompting patterns. | Task description, constraints, examples |
| Engineering Assistant | Apply an LLM/simulator to practical development work. | Code, error message, requirement |
| Strategy Lab | Compare prompting methods on one controlled task. | Common task and evaluation criteria |
| Trace & Metrics | Show prompt, output and simple measurements. | Generated response and runtime data |

## C Problem Statement – Detailed Requirements

### 1. Prompt Studio

The user enters an engineering task and chooses Zero-Shot, Few-Shot, or Structured Reasoning. The system assembles the final prompt and displays it before presenting the generated answer. A few-shot option accepts small examples that demonstrate the desired response format.

### 2. Engineering Assistant Modules

The revised implementation uses three modules: Program Builder, Bug Investigator, and Specification Writer. Program Builder generates a solution outline and code explanation; Bug Investigator identifies likely causes and suggests verification steps; Specification Writer converts informal requirements into organized software documentation.

### 3. Strategy Lab

A single task is executed with each strategy using the same base information. The interface records approximate token count, elapsed time, and a rubric-based quality score. This makes the comparison more meaningful than judging one isolated response.

**4. Responsible Use**

The system warns users not to enter passwords, personal records, proprietary source code, or other sensitive information. Generated code is presented as assistance rather than guaranteed-correct software, and users are encouraged to test important outputs.

**Requirements and Constraints**

| Category | Revised Requirement |
|---|---|
| Functional | Prompt construction, three task modules, comparison dashboard, validation messages |
| Technology | Python and Streamlit with a replaceable model adapter |
| Performance | Interactive response suitable for classroom demonstration |
| Reliability | Empty-input checks, exception handling, deterministic simulator fallback |
| Security | No hard-coded API credentials and no unnecessary sensitive input |
| Resources | Simple architecture that can run on a student laptop |

**D Student Work**

**5. Problem Understanding and Formulation**

PromptCraft is designed as a learning-oriented engineering assistant. Instead of hiding the prompt behind a single chat box, the application separates task description, strategy selection, prompt construction, model execution, and evaluation. This separation helps users understand the relationship between prompt structure and output behavior.

The primary inputs are task descriptions, optional examples, source code, bug information, and software requirements. The main outputs are a constructed prompt, generated response, elapsed execution time, approximate token count, and a quality indicator.

**6. Objectives**

• Create a simple interface for experimenting with three prompting approaches.

• Demonstrate prompt construction for realistic software-engineering activities.

• Provide reusable modules rather than one large prompt.

• Compare response quality and efficiency using consistent test tasks.

• Encourage verification and responsible use of generated technical content.

**7. Expected Outcomes**

At completion, the prototype should allow a student to select a strategy, inspect the generated prompt, run an engineering task, and review basic metrics. It should also provide a comparison table that makes the strengths and limitations of each strategy visible.

**8. Assumptions**

The prototype is intended for non-sensitive classroom examples. The LLM layer may be replaced by an approved API or retained as a simulator when internet access or API credentials are unavailable. Results are considered demonstrations and require human verification.

## 3Application of Relevant Course Knowledge / Concepts

### Prompt Engineering Concepts

**Zero-Shot:** Gives the model a direct instruction without examples. It is compact and useful for straightforward tasks.

**Few-Shot:** Adds representative input-output examples. It is useful when the expected structure, terminology, or style needs to be demonstrated.

**Structured Reasoning:** Requests a systematic analysis with explicit checks, assumptions, and edge cases. The application records only the final response rather than requiring hidden reasoning to be exposed.

### LLM Integration Concept

The design separates prompt generation from response generation through a small model-adapter function. A real provider can therefore replace the simulator without changing the Streamlit interface or task modules.

### Software Engineering Concepts

The prototype applies modular functions, reusable prompt templates, input validation, exception handling, test cases, and measurable evaluation. These practices make the application easier to maintain and easier to demonstrate during assessment.

### Evaluation Design

A controlled evaluation keeps the engineering task constant while changing the prompting strategy. The response is then judged using a small rubric covering correctness, completeness, clarity, and usefulness. Runtime and approximate token count are recorded separately so quality is not confused with efficiency.

### Design Principle

The key principle is transparency: every generated result should be traceable to a visible prompt template and user-provided task. This supports learning, debugging of prompts, and responsible adoption of LLM-assisted engineering.

## D. Solution / Design / Methodology

The revised architecture follows a modular pipeline:

**User Interface □Input Validator □Prompt Template □Model Adapter □Output Checker □Metrics Engine □**

**Results View**

**Component Responsibilities**

| Component | Responsibility |
|---|---|
| User Interface | Collect task, strategy, examples and module selection. |
| Input Validator | Reject blank input and normalize basic text fields. |
| Prompt Template | Create strategy-specific and module-specific instructions. |
| Model Adapter | Call the simulator or approved LLM service. |
| Output Checker | Check for empty or obviously incomplete responses. |
| Metrics Engine | Estimate tokens, measure latency and calculate rubric score. |
| Results View | Display prompt, response, metrics and comparison table. |

**Application Workflow**

• Collect the engineering task.

• Select a prompting strategy or task module.

• Construct the transparent prompt.

• Execute the simulator/API through the model adapter.

• Check the returned text for basic validity.

• Calculate quality, latency and token estimates.

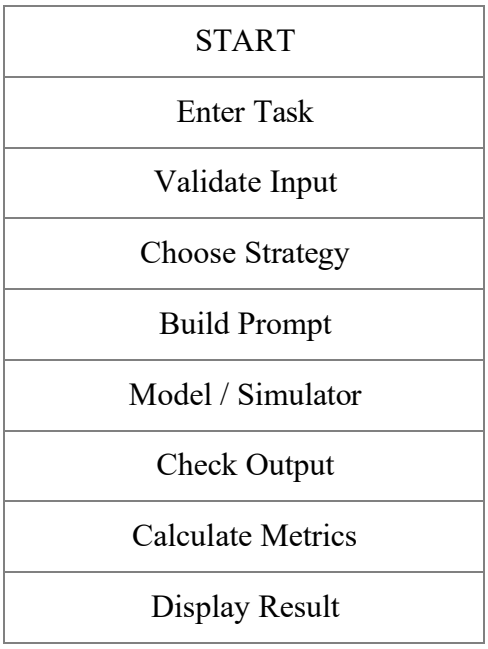
• Display the result and allow comparison.

### E.Algorithm / Pseudocode / Flowchart

**Algorithm: PromptCraft Evaluation**

- Start the application.
- Read the engineering task and optional examples.
- Validate that the task is not empty.
- Select a prompt strategy.
- Build the corresponding prompt template.
- Send the prompt to the model adapter.
- Measure elapsed time and estimate token usage.
- Check the response and calculate a quality score.
- Repeat the process for the remaining strategies when comparison is requested.
- Present the metrics in a comparison table.
- Allow the user to interpret the quality–efficiency trade-off.
- Stop.

**Flowchart**



| Compare Strategies |
| --- |
| END |

**Quality Score Idea**

The prototype can assign a simple score from 0–100 using correctness, completeness, clarity and relevance. The score is a classroom indicator, not a formal benchmark. For a production system, evaluation should use a stronger rubric and task-specific automated checks.

## E. Implementation / Source Code – Revised Version

```
import streamlit as st
import time

st.set_page_config(page_title="PromptCraft V2", layout="wide")
st.title("PromptCraft – Engineering Assistant")

def make_prompt(task, strategy,
    examples=""): if strategy == "Zero-
    Shot":
        return f"Act as a software engineer.\nTask: {task}\nGive a concise, testable
    answer." if strategy == "Few-Shot":
        return (f"Act as a software
                engineer.\nExamples:\n{examples}\n" f"Task:
                {task}\nFollow the demonstrated structure.")
    return (f"Act as a software engineer.\nTask: {task}\n"
            "Use a structured checklist: assumptions, approach, edge cases, tests.")

def
    model_adapter(prom
    pt): text =
    prompt.lower()
    if "debug" in text or "bug" in text or "error" in text:
        return "Check the failing input, isolate the faulty condition, apply a minimal fix, and retest edge
    cases." if "requirement" in text or "document" in text:
        return "Document the objective, actors, inputs, workflow, constraints, acceptance criteria, and
    tests." return "Provide a clear solution, explain the main steps, mention edge cases, and suggest
    validation tests."

def evaluate(task, strategy, examples=""):
    prompt = make_prompt(task, strategy,
    examples) start = time.perf_counter()
    answer = model_adapter(prompt)
    elapsed = time.perf_counter() -
```

```
    start
    tokens = len((prompt + " " +
    answer).split()) score = min(100, 68 +
    len(answer) // 12) return prompt, answer,
    elapsed, tokens, score
```

The revised implementation uses a model-adapter function so that the simulated response can later be replaced by a real provider. The scoring function is intentionally simple for classroom demonstration.

**Implementation / Source Code – Interface and Modules**

```
strategy = st.sidebar.selectbox(
    "Prompt Strategy", ["Zero-Shot", "Few-Shot", "Structured Reasoning"]
)

task = st.text_area("Engineering Task",
    "Create a Python function to check whether a password is strong.")

examples = ""
if strategy == "Few-Shot":
    examples = st.text_area("Examples",
        "Input: short password -> Output: reject and explain minimum length")

if  st.button("Generate"):
    if not task.strip():
        st.warning("Please enter an engineering task.")
    else:
        p, r, t, tok, q = evaluate(task, strategy, examples)
        st.subheader("Constructed Prompt")
        st.code(p)
        st.subheader("Response")
        st.write(r)
        st.write(f"Quality: {q}/100 | Time: {t:.4f}s | Tokens: {tok}")

st.divider()
st.header("Task
```

```
Modules")
module = st.selectbox("Select Module", [
    "Program Builder", "Bug Investigator", "Specification Writer"
])
artifact = st.text_area("Artifact / Requirement")

if st.button("Run Module") and artifact.strip():
    if module == "Program Builder":
        task2 = "Generate and explain a program for: " + artifact
    elif module == "Bug Investigator":
        task2 = "Debug this bug and propose tests: " + artifact
    else:
        task2 = "Write software documentation from this requirement: " +
    artifact p, r, t, tok, q = evaluate(task2, "Zero-Shot")
    st.code(p)
    st.write(r)
```

**Why this version is different**

The module names and task framing have been changed from the earlier version. The interface now uses Program Builder, Bug Investigator and Specification Writer, while the structured reasoning option emphasizes assumptions, edge cases and tests rather than exposing hidden reasoning.

## H. Modern Tools / Test Plan

### Tools Used

| Tool / Technique | Use in the project |
|---|---|
| Python | Application logic, prompt templates and evaluation functions |
| Streamlit | Web interface and interactive controls |
| LLM API / Simulator | Response generation layer |
| Browser | Run and demonstrate the application |
| Tabular evaluation | Compare strategy metrics |

### Test Cases

| ID | Test Input | Expected Behaviour |
|---|---|---|
| T1 | Create a password-strength function | Prompt and response are displayed; output includes validation logic. |
| T2 | Undefined variable in a loop | Bug Investigator identifies the likely fault and suggests a retest. |
| T3 | Password-reset requirement | Specification Writer returns structured software documentation. |
| T4 | Empty task field | Application shows a warning without crashing. |
| T5 | Same optimization task with all strategies | Comparison table contains quality, latency and token estimates. |

### Validation Approach

Every module is tested first with normal input, then with empty or incomplete input. The comparison experiment uses the same task statement for all strategies to reduce experimental bias. Runtime values are collected during execution rather than being hard-coded into the final result.

## I. Results and Validation – Revised Evaluation

The following values are sample demonstration results for the revised version. Actual runtime measurements should be captured when the application is executed.

| Strategy | Quality / 100 | Typical Prompt Size | Observed Behaviour |
|---|---|---|---|
| Zero-Shot | 78 | Low | Fast and suitable for direct, well defined tas |
| Few-Shot | 86 | Medium | More consistent when a desired format is sh |
| Structured Reasoning | 89 | Medium–High | Better suited to tasks requiring checks and |

### Module Validation Summary

| Module | Input | Output Check | Status |
|---|---|---|---|
| Program Builder | Algorithm request | Contains implementation idea and testing notes | Pass |
| Bug Investigator | Bug description | Includes cause, correction and retest guidance | Pass |
| Specification Writer | Informal requirement | Contains objective, requirements and acceptance crit | ePriaass |

### Interpretation

The comparison suggests that no single prompting strategy is universally best. Direct tasks benefit from compact prompts, while examples improve consistency when output structure matters. More explicit reasoning instructions can improve completeness but may increase prompt length. The best choice therefore depends on the task and the evaluation criteria.

## J. Analysis, Engineering Decision and Broader Considerations

### Engineering Decision

For routine code requests, Zero-Shot is an efficient default. For strict output formats, Few-Shot is preferred because examples communicate the expected structure. For debugging and requirement analysis, Structured Reasoning is useful because it encourages assumptions, edge cases and verification steps. The application therefore selects strategies according to task characteristics rather than enforcing one universal template.

### Trade-Off Analysis

| Factor | Zero-Shot | Few-Shot | Structured Reasoning |
|---|---|---|---|
| Prompt length | Low | Medium | Medium–High |
| Setup effort | Low | Medium | Medium |
| Format consistency | Moderate | High | High |
| Complex-task support | Moderate | High | High |
| Potential latency | Low | Medium | Medium–High |

### Responsible AI

The system should communicate that generated content may contain mistakes. Users should verify code by execution and review documentation against the original requirement. API credentials must remain outside source code, and sensitive or confidential information should not be entered into classroom demonstrations.

### Limitations

The simulator does not represent the full behaviour of a modern LLM. Token counts are approximate, and the simple quality score cannot replace expert evaluation. Future versions should use a real approved model, task-specific automated tests, and a larger benchmark set.

### Future Enhancements

Useful extensions include experiment history, downloadable reports, model selection, configurable evaluation rubrics, code execution in a sandbox, and a prompt library organized by software-engineering task.

## K. Conclusion and Reflection

### Conclusion

The revised PromptCraft prototype demonstrates how prompt engineering can be integrated with common software-development activities in a single educational application. It provides visible prompt construction, modular engineering assistance, strategy comparison, basic metrics, and responsible-use guidance.

The solution remains lightweight because it uses Python and Streamlit and avoids unnecessary database or infrastructure complexity. Its modular model-adapter design also allows a simulator to be replaced by an approved LLM API when required.

### Reflection

The project demonstrates that effective LLM usage is not simply a matter of asking a question. Prompt structure, examples, constraints, verification steps, and evaluation criteria all influence the usefulness of the result. Treating prompts as reusable engineering artifacts makes experimentation more systematic and reproducible.

### Key Learning Outcomes

• Applied multiple prompt-engineering strategies to practical tasks.

• Built reusable templates instead of repeating prompt text manually.

• Connected an interface to a model abstraction layer.

• Compared output quality and efficiency using a controlled task.

• Considered hallucination, privacy, verification and secret management.

### Submission Evidence Checklist

■ Working Streamlit source code

■ Screenshots of Prompt Studio and task modules

■ Strategy comparison results

■ Test-case outputs

■ Responsible-AI notes

■ Repository or source-code reference

## L. Assessment Rubric and CO–PO Mapping – Revised Presentation

| Assessment Criterion | Marks |
|---|---|
| Problem understanding and formulation | 10 |
| Application of prompt engineering and LLM concepts | 20 |
| Solution design and implementation | 20 |
| Modern tool usage | 10 |
| Results, testing and validation | 15 |
| Analysis, trade-offs and justification | 15 |
| Broader considerations and professional responsibility | 5 |
| Technical documentation and reflection | 5 |
| TOTAL | 100 |

**CO–PO–Assessment Mapping**

| Assessment Component | CO | PO(s) | Bloom Level | Marks |
|---|---|---|---|---|
| Problem formulation | CO1/CO2 | PO1, PO2 | L3 | 10 |
| Prompt engineering techniques | CO1/CO2 | PO1, PO2, PO3 | L3/L4 | 8 |
| LLM task module integration | CO1 | PO1, PO2, PO3 | L3/L4 | 6 |
| Prompt evaluation and comparison | CO2 | PO1, PO2, PO3 | L3/L4 | 6 |
| Integrated PromptCraft solution | CO1–CO2 | PO2, PO3, PO5 | L4/L5/L6 | 20 |
| Modern tools | CO1–CO2 | PO5 | L3 | 10 |
| Validation | CO1–CO2 | PO4 | L4/L5 | 15 |
| Analysis and trade-offs | CO1–CO2 | PO2, PO4 | L4/L5 | 15 |
| Broader considerations and reflection | CO1–CO2 | PO6, PO12 | L3/L4 | 10 |

**Faculty Evaluation / Continuous Improvement**

Performance Level 3: ☐70% | Level 2: 60–69% | Level 1: 50–59% | Level 0: <50%

Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action for next offering: _____

Note: PO mappings are presented as an assessment-format reference and should be verified against the current course matrix before submission.